



BURSA ULUDAĞ ÜNİVERSİTESİ

Fen Edebiyat Fakültesi

Matematik Bölümü

**PROJE: Otonom Araçlarda Fren Mesafesi: Türev ve
Değişim Hızı Analizi**

Grup No: 3

082140016 İlayda ŞEN

082140022 Fatma AKINCI

082140028 Gülenaz BAYLAN

1. Yöntem:

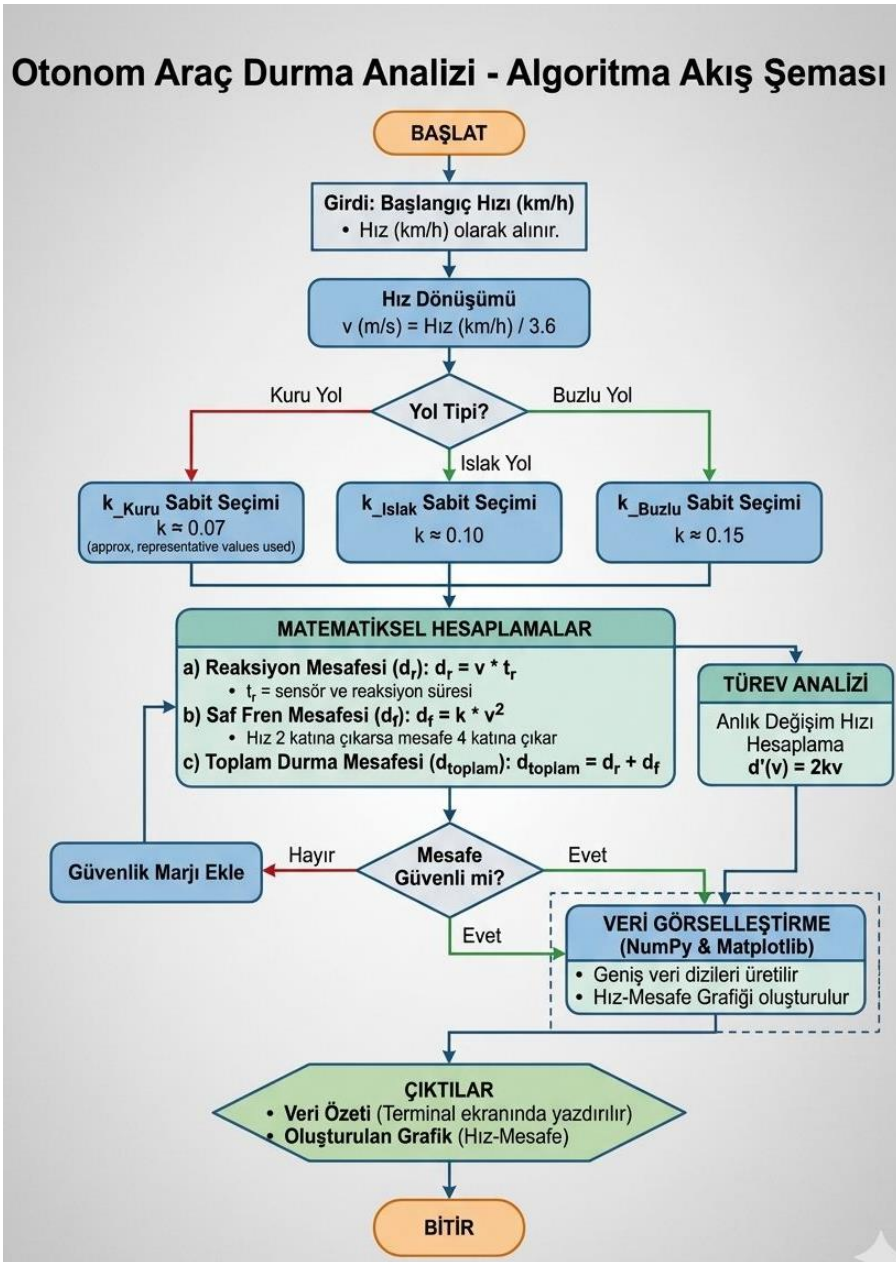
Projemiz, verinin girişinden görselleştirmesine kadar dört temel aşamada çalışacaktır:

- Birinci adımda, kullanıcıdan aracın ilk hızını öğreniriz. Günlük hayatta hız km/h olarak düşünürüz, bu yüzden girişi bu şekilde isteriz. Matematiksel modellerde ve fizik formüllerinde işlem yaparken m/s cinsinden kullanmamız gerekir.[3] Çünkü fren mesafesini kilometre değil metre cinsinden hesaplamak istiyoruz. Bu yüzden kullanıcıdan aldığımız hızı 3.6 rakamına bölerek saniyede kaç metre yol gittiğimizi buluruz. Fakat sadece hız yetmez, aracın o an kuru bir yolda mı ıslak bir yolda mı olduğunu sisteme girmemiz gerekir. Bu formülümüzdeki k sabitini seçmemizi sağlıyor.
- İkinci adımda, hazırladığımız hız verilerini alıp farklı matematiksel fonksiyonlara atıyoruz. Sadece tek bir sonuç bulmak yerine, güvenli bir sürüş için gereken tüm parçaları ayrı ayrı hesaplıyoruz. [4]
 - ✓ Reaksiyon Mesafesini Bulma: Burada $d_r(v) = v \cdot t_r$ kullanıyoruz. Otonom aracın sensörlerinin tehlikeyi algılayıp frene basana kadar geçen sürede aldığı yolu hesaplıyoruz. Bu aşamada hızla yol doğru orantılı artıyor.[1]
 - ✓ Fren Mesafesinin Hesaplanması: Burada $d = k \cdot v^2$ kullanıyoruz. Buradaki hız iki katına çıkarsa durma mesafesinin dört katına çıkacağını görüyoruz.[5]
 - ✓ Toplam Durma Mesafesi: Son olarak, hem reaksiyon mesafesini hem de saf fren mesafesini toplayarak aracın tam olarak nerede durabileceğini buluyoruz.
- Üçüncü adımda, sadece mesafeyi bilmek otonom bir sistem için yeterli değildir, o zaman hızdaki küçük bir artışın mesafeyi ne kadar şiddetli etkilediğinin bilinmesi gerekir. Bu aşamada mesafe fonksiyonunun hızla göre birinci türev alınarak anlık değişim hızı hesaplanır. Bu analiz, sistemin yüksek hızlarda neden daha agresif bir güvenlik payı bırakması gerektiğini matematiksel olarak ispatlar. Türev denklemi $d'(v) = 2k \cdot v$. [6]
- Dördüncü adımda, NumPy ile üretilen geniş veri dizileri Matplotlib kütüphanesine aktarılarak hız-mesafe grafiği oluşturulur ve otonom sistemin güvenliği takip mesafesi sınırları belirlenir.[2]

2. Algoritma Akış Şeması:

- Başlat: Programın çalışmasını başlatır.
- Girdi: Kullanıcı program üzerinden otonom aracın km/h cinsinden hızını girecektir.
- İşlem: Alınan km/h verisi, fiziksel formülleri metre bazında doğru sonuç vermesi için 3.6 değerine bölünerek m/s birimine çevrilir.

- **Hesaplama:** Belirlenen sabit katsayı yardımıyla $d(v)$ ve $d'(v)$ formülleri üzerinden fren mesafesi ve türev işlemleri yapılacaktır. Reaksiyon mesafesi, aracın tehlikeyi algılama süresindeki kat ettiği yol hesaplanır. Fren mesafesi, karesel fonksiyon kullanılarak saf frenleme mesafesi hesaplanır. Türev analizi, hızdaki değişimin mesafeye etkisi anlık değişim hızı ile bulunur. Toplam mesafe, reaksiyon ve fren mesafeleri toplanır.
- **Çıktı:** Reaksiyon mesafesi, fren mesafesi, ve toplam durma mesafesi ayrı ayrı raporlanır. Sonuç olarak terminal ekranında yazdırılır.



3. Uygulama Tasarımı:

- Kullanıcı ne girecek?

Kullanıcı, otonom aracın o anki hız değerini km/h cinsinden sisteme girecektir. Program, kullanıcı dostu bir arayüz üzerinden bu veriyi alır. Ayrıca yazılım mimarisi, ihtiyaç duyulması halinde sistemin reaksiyon süresini (t_r) ve yol sürtünme katsayısını (k) parametrik olarak değiştirmeye uygun tasarlanmıştır; ancak varsayılan olarak bu değerler otonom sistem standartlarına göre sabitlenmiştir.

- Program ne hesaplayacak?

Girdi alındığı anda program şu dört temel matematiksel işlemi eş zamanlı olarak yürütür.

1. Birim Dönüşümü: Analiz prensipleri gereği km/h cinsinden alınan hız m/s birimine çevrilir.
2. Reaksiyon ve Fren Mesafesi: Yazılımın karar verme süresince kat edilen doğrusal mesafe ile frenleme anındaki karesel mesafe ayrı ayrı hesaplanır.
3. Toplam Durma Mesafesi: Bu iki değer toplanarak aracın tam durma noktasına kadar olan $d(v)$ mesafesi bulunur.
4. Türev: Fonksiyonun türevi olan $d'(v) = t_r + 2kv$ denklemi kullanılarak hızdaki bir birimlik artışın durma mesafesi ne kadar etkilediği hesaplanır.

- Sonuç nasıl gösterilecek?

Hesaplamalar tamamlandığında sonuçlar iki farklı formatta gösterilir.

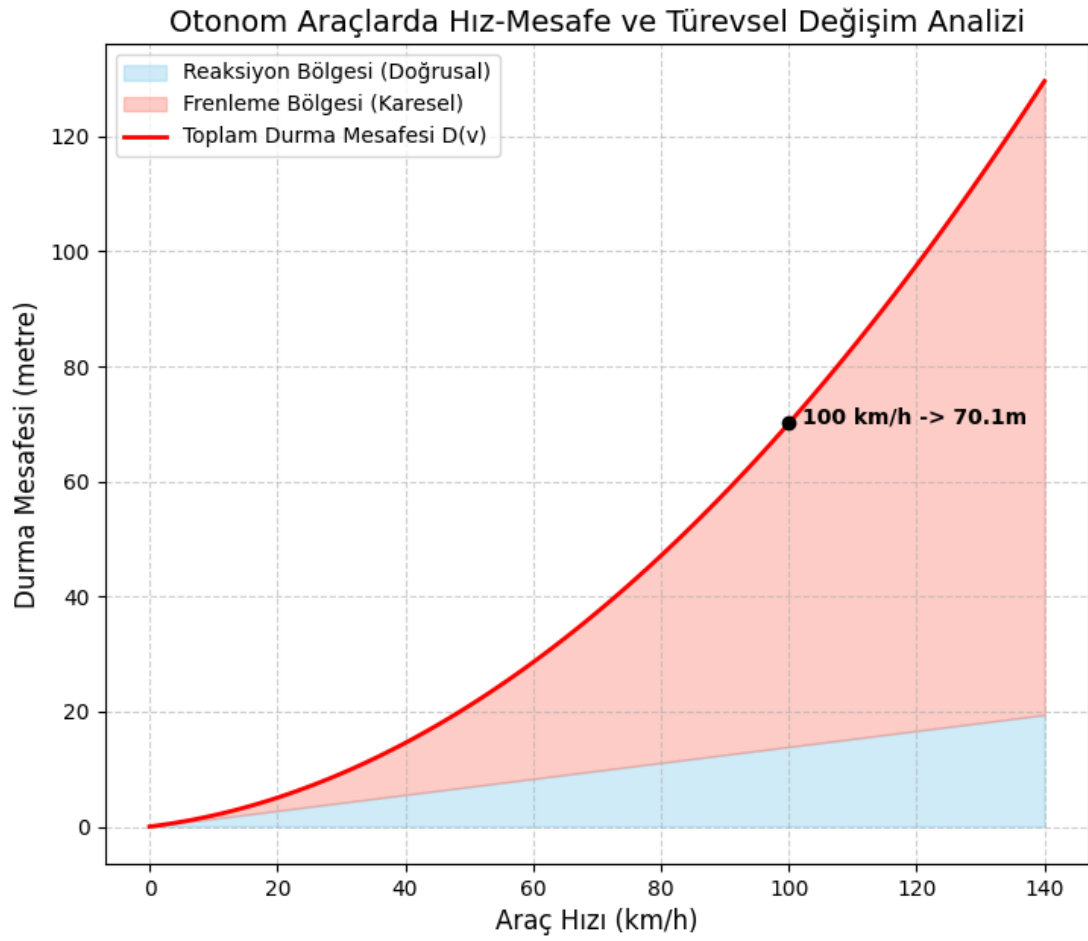
1. Sayısal Özet (Konsol Çıktısı): Reaksiyon mesafesi, fren mesafesi ve toplam durma mesafesi metre cinsinden; değişim oranı ise türev değeri olarak ekranda listelenir.[7]
2. Görsel Analiz (Grafik): Matplotlib kütüphanesi aracılığıyla oluşturulan bir grafikte, hız-mesafe eğrisi çizilir. Kullanıcının girdiği hız değeri bu eğri üzerinde özel bir nokta ile işaretlenir; böylece kullanıcı, hızının güvenlik üzerindeki etkisini görsel olarak analiz edebilir.[10]

4. Başarı Ölçütleri:

- Matematiksel Doğruluk: Python çıktılarının, kâğıt üzerindeki türev hesaplamalarıyla tam olarak örtüşmesi.
- Hata Payı Yönetimi: Yazılımın, sensör sapmalarını temsil eden %10 emniyet payını hesaplamalara hatasız yansıtması.[9]
- Görsel Kanıt: Grafikte hız arttıkça durma mesafesinin parabolik (karesel) artışının net bir şekilde gözlemlenmesi.
- Modüler Yapı: Kodun; hesaplama, dönüşüm ve grafik çizimi gibi işlemleri birbirinden bağımsız fonksiyonlarla kararlı bir şekilde yürütmesi.[8]
- Analitik Yorum: Üretilen türev değerinin, hızdaki artışın güvenliği ne kadar riskli hale getirdiğini bilimsel olarak kanıtlaması.

```
Run GrupNo3 x
C:\Users\user\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\user\PyCharmMiscProject\GrupNo3.py
Hız (km/h) | Toplam Mesafe (m) | Emniyetli Mesafe (m) | Türev (Hassasiyet)
-----
30 | 9.22 | 10.15 | 1.71
50 | 20.99 | 23.09 | 2.52
70 | 37.26 | 40.99 | 3.33
90 | 58.02 | 63.82 | 4.14
110 | 83.28 | 91.61 | 4.95
130 | 113.03 | 124.34 | 5.76
Process finished with exit code 0
```

Python'da çalışan ilk prototip kod ve örnek senaryonun çıktısı.



KAYNAKÇA

[1] Hibbeler, R. C. (2016). Engineering mechanics: Dynamics (14. baskı). Pearson Education.

<https://www.pearson.com/en-us/subject-catalog/p/engineering-mechanics-dynamics/P200000003310/9780133915389>

[2] Thrun, S. (2010). Toward robotic cars. Communications of the ACM, 53(4), 99-106. <https://doi.org/10.1145/1721654.1721679>

[3] Giancoli, D. C. (2014). Physics: Principles with applications (7. baskı). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/physics-principles-with-applications/P200000003158/9780321625922>

[4] Knight, R. D. (2016). Physics for scientists and engineers: A strategic approach (4. baskı). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/physics-for-scientists-and-engineers-a-strategic-approach-with-modern-physics/P200000003185/9780133942651>

[5] Stewart, J. (2015). Calculus: Early transcendentals (8. baskı). Cengage Learning. <https://www.cengage.com/c/calculus-early-transcendentals-8e-stewart/9781285741550/>

[6] Thomas, G. B., Weir, M. D., & Hass, J. (2018). Thomas' calculus (14. baskı). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/thomas-calculus/P200000003387/9780134438986>

[7] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585(7825), 357-362.

<https://doi.org/10.1038/s41586-020-2649-2>

[8] Johansson, R. (2018). Numerical Python: Scientific computing and data science applications (2. baskı). Apress.

<https://doi.org/10.1007/978-1-4842-4246-9>

[9] Oliphant, T. E. (2006). A guide to NumPy. Trelgol Publishing.

https://archive.org/details/guide_to_numpy

[10] Anderson, J. M., Kalra, N., Stanley, K. D., Sorensen, P., Samaras, C., & Oluwatola, O. A. (2014). Autonomous vehicle technology: A guide for policymakers. RAND Corporation.

https://www.rand.org/pubs/research_reports/RR443-2.html